

AutoARIMA inside the `mstl_multistep` Model

What it forecasts, how its orders are chosen, and the hyperparameters we set

Model: `config_tgtlag3_var` — `statsforecast 2.0.3`

June 5, 2026

Abstract

In this model an **AutoARIMA is fit independently per location** to the MSTL-deseasonalised log target. It is the *base* forecaster: it supplies both the conditional mean μ_h and the predictive spread σ_h , and its in-sample one-step residual is the target the Random Forest then corrects. We fix a handful of options (non-seasonal, stepwise, approximate, 68% intervals); everything else — the (p, d, q) orders, the coefficients, the innovation variance — is selected automatically per series by the Hyndman–Khandakar algorithm. On this data the chosen models are low-order and near-random-walk: 93% use a single difference ($d=1$), the AR order is usually 0, and the most common forms are ARIMA(0, 1, 1), (0, 1, 0), (0, 1, 2).

1 Role in the pipeline

After MSTL removes seasonality, each location has a deseasonalised log series $D_{\ell,t}$ (trend + remainder). A separate ARIMA is fit to each D_ℓ and does two jobs:

- **Mean.** The h -step forecast $\mu_{\ell,h}$ carries the autoregressive / trend dynamics. Reconstruction adds the seasonal-naive term and the RF correction: $\hat{L} = S + \mu_h + g(x)$.
- **Spread.** The forecast interval gives $\sigma_{\ell,h}$, the model’s principal source of predictive uncertainty (it grows with horizon).

ARIMA is fit **per location** (406 independent models); pooling across locations happens only later, in the Random Forest.

2 What we fix vs. what AutoARIMA finds

We call AutoARIMA (`statsforecast`) with only a few options set; the rest are its defaults.

Option	Value (this model)	Meaning
<code>season_length</code>	1	non-seasonal — MSTL already removed seasonality
<code>stepwise</code>	True	Hyndman–Khandakar stepwise search (not full grid)
<code>approximation</code>	True	approximate (CSS) likelihood during search; final fit by MLE
<code>level</code>	68	predictive interval level used to back out σ_h
<i>inherited defaults that shape the search:</i>		
<code>ic</code>	<code>aicc</code>	model-selection criterion (corrected AIC)
<code>test</code>	<code>kpss</code>	unit-root test that picks the differencing order d
<code>max_p, max_q</code>	5, 5	upper bounds on AR / MA order
<code>max_d</code>	2	max differencing
<code>start_p, start_q</code>	2, 2	starting orders for the stepwise search
<code>allowdrift, allowmean</code>	True, True	permit a drift (when $d=1$) / mean (when $d=0$) term
<code>nmodels</code>	94	cap on models evaluated in the search

Found automatically, per location: the orders (p, d, q) , whether a drift/mean term is included, the AR/MA coefficients, and the innovation variance σ^2 . Because `season.length=1`, the seasonal orders are inert ($P=Q=D=0$).

3 How the orders are chosen (Hyndman–Khandakar)

For each series:

1. **Differencing d :** successive KPSS unit-root tests choose the smallest $d \leq 2$ that makes the series stationary.
2. **Seed models:** fit ARIMA(2, d , 2), (0, d , 0), (1, d , 0), (0, d , 1) (with a constant where allowed) and keep the best by AICc.
3. **Stepwise search:** repeatedly perturb p and q by ± 1 and toggle the constant; accept any change that lowers AICc; stop at a local optimum (or `nmodels`).
4. **Final fit:** with `approximation=True`, the search ranks models by a fast conditional-sum-of-squares likelihood; the chosen model is then re-estimated by full maximum likelihood.

We enable `approximation` purely for speed — the model fits hundreds of ARIMAs per backtest split (406 locations $\times$ 12 splits).

4 Orders actually selected on this data

Sample of 135 locations (deseasonalised log target, training window; 1 fell back to the constant-mean baseline):

order	share	marginal	
ARIMA(0, 1, 1)	20%	$d = 1$	93%
ARIMA(0, 1, 0)	17%	$d = 0$	7%
ARIMA(0, 1, 2)	16%	$p = 0$	53%
ARIMA(1, 1, 1)	11%	$p = 1$	28%
ARIMA(1, 1, 2)	7%	$p \geq 2$	19%
ARIMA(2, 1, 1)	5%	$q = 0$	27%
ARIMA(1, 1, 0)	4%	$q = 1$	44%
others	20%	$q = 2$	27%

A drift/mean term is included in $\sim 16\%$ of series; the innovation variance on the deseasonalised log scale has median $\sigma^2 \approx 0.17$ (IQR 0.13–0.22).

Interpretation. The dominant forms — (0, 1, 1), (0, 1, 0), (0, 1, 2) with $d=1$ and AR order usually 0 — are essentially a *differenced local-level* process (a random walk, often with light MA smoothing; ARIMA(0, 1, 1) is exactly simple exponential smoothing). This is why, beyond one step, the mean forecast μ_h flattens toward the last estimated level rather than mean-reverting strongly — and why the Random Forest’s covariate correction, not ARIMA’s AR term, carries most of the multi-step “signal.”

5 Extracting the mean and the spread

`AutoARIMA.forecast` returns the h -step mean and a 68% interval $[\text{lo}, \text{hi}]$. We treat the predictive distribution as Gaussian and back out the standard deviation:

$$\sigma_{\ell,h} = \frac{\text{hi}_{\ell,h} - \text{lo}_{\ell,h}}{2z}, \quad z = \Phi^{-1}\left(0.5 + \frac{68}{200}\right) = \Phi^{-1}(0.84) \approx 0.994.$$

The 68% level is chosen so that $z \approx 1$, i.e. the interval half-width is essentially one standard deviation. $\sigma_{\ell,h}$ is clamped to be finite and positive (floor 10^{-3}) and *grows with h* . In the model the final spread is widened further by the variance head, $\sigma_{\text{eff}}^2 = \sigma_h^2 + \frac{1}{2}v(x)$.

6 Input handling and fallbacks

Each series is cleaned before fitting: linear interpolation of internal gaps, then forward/backward fill (`interpolate(limit_direction="both").ffill().bfill()`), so AutoARIMA always sees a finite series. Two safeguards:

- a series with *no* finite values (a location with no history in the window) gets a zero-mean, unit- σ baseline;
- if AutoARIMA raises, it falls back to a constant-mean / sample-standard-deviation forecast.

7 The residual the Random Forest consumes

The RF target is the ARIMA **in-sample, one-step-ahead** residual

$$R_{\ell,t} = D_{\ell,t} - \hat{A}_{\ell,t}, \quad \hat{A}_{\ell,t} = \mathbb{E}[D_{\ell,t} \mid D_{\ell,<t}] \text{ (one-step fitted).}$$

Provenance note. At training the residual is computed against the *in-sample, single-step* fitted values; at prediction the RF correction is added to the *out-of-sample, h-step* forecast μ_h . So the residual the RF learns and the error it implicitly corrects differ in both senses (in-sample vs out-of-sample, single- vs multi-step). This drift is mild at short horizon (the near-random-walk models change little over $h=3$) and grows with h ; the `rf_horizon_feature` variant trains on genuine out-of-sample multi-horizon residuals to remove it, and empirically helps only at long horizon.